

PagedAttention 技术调研报告

PagedAttention 技术调研报告

SOSP 2023 | vLLM 项目 (83k nodes / 490k edges) | 2026-06-29

1. 背景：LLM 推理为什么慢

1.1 自回归生成

大语言模型（如 GPT、LLaMA）生成文本的方式是逐 token 生成：

输入："法国的首都是"
→ token_1: "巴"
→ token_2: "黎"
→ token_3: "<end>"

每生成一个新 token，模型需要“回顾”之前所有 token 的信息。这些信息存储在 **KV Cache** 中。

1.2 KV Cache 是什么

Transformer 的 attention 机制中，每个 token 会生成一对向量：- **K (Key)**：用于和其他 token 做匹配 - **V (Value)**：token 的实际内容表示

生成第 N 个 token 时，需要前 N-1 个 token 的 K 和 V。如果每次都要重新计算，计算量是 $O(N^2)$ 。KV Cache 就是把已计算的 K 和 V 存下来，新 token 只需要算自己和历史的 attention。

KV Cache 的大小：一个 13B 参数的模型，对单个请求（2048 tokens），KV Cache 约需 **1.7GB** 显存。

1.3 现有系统的浪费

以 13B 模型在 NVIDIA A100 (40GB) 上运行为例：

显存占用	比例	说明
模型权重	~65%	固定不变
KV Cache	~30%	动态变化，是瓶颈
其他	~5%	激活值等临时数据

R1	R2	R3	R1	R2		R3	R1	R3	R2			..
----	----	----	----	----	--	----	----	----	----	--	--	----

R1 用了 3 个 block (48 tokens)
R2 用了 3 个 block
R3 用了 3 个 block

空闲 block 可以分配给任何新请求
利用率：96%+

2.3 Block Table：把逻辑映射到物理

每个请求有一个 **block table**，类似操作系统的页表：

请求 A 的 block_table: [3, 7, 12]
逻辑上: block_0 → 物理 block #3
 block_1 → 物理 block #7
 block_2 → 物理 block #12

请求 B 的 block_table: [1, 5]
逻辑上: block_0 → 物理 block #1
 block_1 → 物理 block #5

关键：逻辑上连续的 token 序列，在物理上可以分散在不同的 block 中。
GPU 做 attention 计算时，通过 block table 找到真正的物理 block 位置。

2.4 PagedAttention 算法的四个好处

好处	解释
消除内部碎片	不再预分配最大长度，按需分配 block
消除外部碎片	所有 block 一样大，不存在”空隙”
内存共享	两个请求的相同前缀可以共享同一组 block
灵活调度	请求被中断时可以释放 block，恢复时重新分配

3. Block 内存管理

3.1 三种状态

每个物理 block 处于三种状态之一：

Block Pool（所有物理 block 的池子）

Free Blocks [空闲]	等待被分配给任何请求
Allocated Blocks [已分配]	已被某个请求使用，存储该请求的 K 和 V

请求完成 → 释放回 Free

— Cached Blocks [已缓存]

存储的内容被 hash 记录

新请求如果有相同内容 → 直接复用 (prefix caching)

3.2 Prefix Caching

很多请求共享相同的前缀。比如：

请求 A: "请帮我翻译下面这段话：Hello World" ← prefix

请求 B: "请帮我翻译下面这段话：Good Morning" ← 相同 prefix!

请求 C: "请帮我翻译下面这段话：How are you" ← 相同 prefix!

三个请求的前 8 个 token 完全相同。vLLM 会：1. 第一个请求 (A) 生成时，计算 prefix 的 hash 并缓存 2. 后续请求 (B、C) 检测到相同 hash → 直接复用已缓存的 block 3. 只分配新 token 需要的 block

Hash 如何计算：对 block 内的 token 序列计算 hash 值。当新请求的某个 block 内容与缓存中某 block hash 相同时，判定为命中 (hit)。

LRU 淘汰：缓存空间有限时，淘汰最久未使用的 block。被淘汰的 block 回到 Free 状态。

效果：对于 chatbot、翻译等有长 system prompt 的场景，prefix caching 可以节省大量重复计算。论文实验显示，在某些 workload 下可以减少 50% 以上的显存占用。

3.3 抢占 (Preemption)

当 GPU 显存不够分配新 block 时，vLLM 可以“抢占”正在运行的请求：

显存不足 → 选择一个正在运行的请求

→ 释放它占用的 block (回到 Free pool)

→ 把请求放回等待队列

→ 等有资源时重新执行

论文提出了两种释放策略：- **交换 (Swap)**：把 block 内容拷贝到 CPU 内存，恢复时拷贝回来。恢复快，但需要 CPU 内存 - **重计算 (Recompute)**：直接丢弃 block 内容，恢复时重新算一遍。省内存，但恢复慢

4. 一次完整的推理流程

4.1 Prefill 阶段（首次推理）

当用户请求到达时，vLLM 需要处理输入的 prompt：

输入："请帮我翻译下面这段话"（8 tokens）

Step 1: 计算需要几个 block ($8 \text{ tokens} / 16 \text{ tokens_per_block} = 1 \text{ block}$)
Step 2: 从 Free Pool 分配 1 个物理 block (比如 block #3)
Step 3: 更新 block_table: [3]
Step 4: 将 8 个 token 的 K 和 V 写入 block #3
Step 5: 计算所有 8 个 token 之间的 attention (一次矩阵乘法)
Step 6: 生成第 9 个 token

4.2 Decode 阶段（逐 token 生成）

当前状态：已生成 8 个 token, block_table = [3]

Step 1: 生成第 9 个 token, 需要第 9 个位置
Step 2: block #3 还有空位 ($16-8=8$), 继续使用
Step 3: 计算新 token 对历史 8 个 token 的 attention
Step 4: 将新 token 的 K、V 写入 block #3 的下一个位置

...若干步后, block #3 满了...

Step N: block #3 满 ($16/16$), 需要新 block
Step N+1: 从 Free Pool 分配 block #7
Step N+2: 更新 block_table: [3, 7]
Step N+3: 新 token 的 K、V 写入 block #7

4.3 多请求并发（Continuous Batching）

时间 →

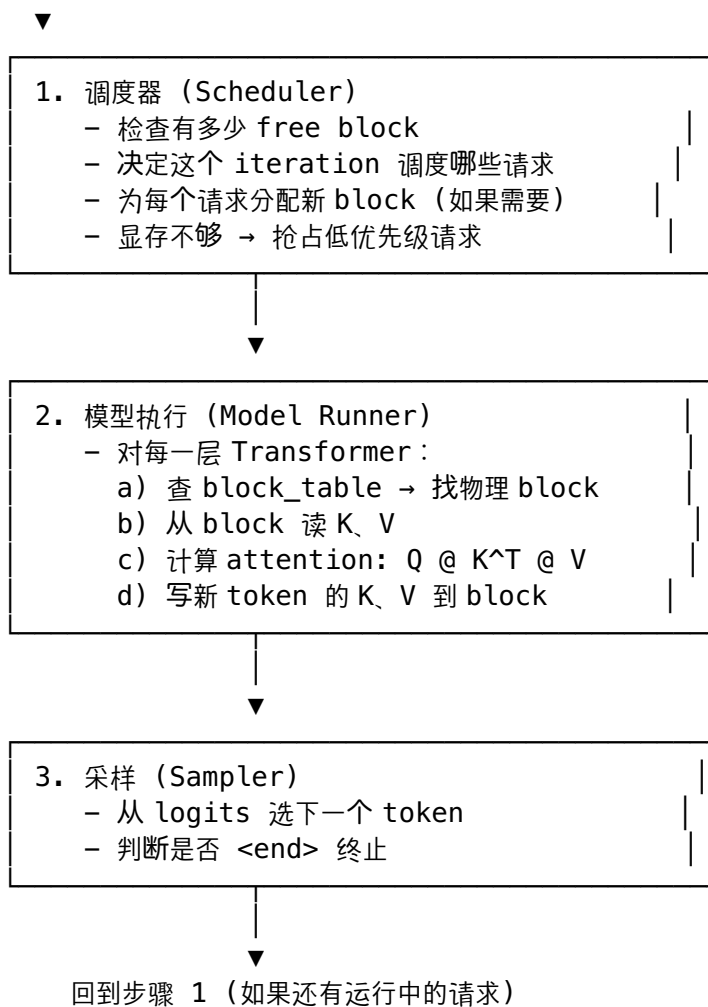
请求A:  (已完成, 释放 block 3,7,12)
请求B:  (运行中, 占用 block 1,5,8,15)
请求C:  (新加入, 分配 block 2,9)
请求D:  (新加入, 复用 cached block 6)

↑ ↑
A完成,释放资源 C、D随时加入, 不等B

关键：不同于传统 batching（等所有请求完成才下一批），vLLM 的请求可以随时加入、随时退出。这称为 **iteration-level scheduling** ——每次 iteration（生成一个 token）都是一个调度决策点。

4.4 完整流程图

用户请求
|



5. 工程实现

5.1 代码规模

vLLM 代码库共有 83,139 个可分析节点，主要分布：

模块	文件位置	职责
调度器	vllm/v1/core/sched/ scheduler.py	737 行，决定每个 iteration 调度哪些请求
Block 管理	vllm/v1/core/ kv_cache_manager.py	管理 free/ allocated/cached 三种 block 状态
CUDA kernel	csrc/libtorch_stable/ attention/	NVIDIA GPU 上的 PagedAttention 实 现
ROCm kernel	csrc/rocm/attention.cu	

模块	文件位置	职责
Python 接口	vllm/v1/attention/ops/ paged_attn.py	AMD GPU 上的 PagedAttention 实 现
		提供 PagedAttention 类 供上层调用

5.2 CUDA Kernel 设计要点

GPU 上运行 PagedAttention 的核心 kernel 函数有 22 个参数，其中最关键的是：

- **block_tables**：块表，逻辑 block → 物理 block 的映射
- **key_cache / value_cache**：两个大张量，分别存储所有物理 block 的 K 和 V
- **seq_lens**：每个请求当前已生成的 token 数量
- **block_size**：每个 block 能存几个 token（通常是 16）
- **scale**：attention 的缩放因子 ($1/\sqrt{d_k}$)

Kernel 的 Grid 结构是 (num_heads, num_seqs, num_partitions)，三维并行：
- num_heads：每个 attention head 独立计算
- num_seqs：每个请求独立计算
- num_partitions：每个 partition 处理一部分 token，最后 reduce 汇总

5.3 双平台支持

vLLM 在 NVIDIA 和 AMD GPU 上都能运行。AMD 平台使用 ROCm (HIP) 重写了 CUDA kernel，代码在 `csrc/rocm/attention.cu`。平台选择通过 `use_rocm_custom_paged_attention()` 函数自动判断。

6. 实验结果

指标	传统系统	vLLM (PagedAttention)
KV Cache 内存利用率	20-38%	96%+
吞吐量	基线	2-4× 提升
长序列优势	—	序列越长，优势越明显
Beam search 加速	—	多个 beam 共享 KV cache block

7. 相关项目

 PagedAttention 论文 (SOSP 2023)

-  [vllm-project/vllm](#) 主引擎实现
-  [vllm-project/vllm-ascend](#) Ascend NPU 官方后端
-  [cosdt/vllm-ascend](#) 团队维护版 Ascend 后端
-  [cosdt/vllm-ascend-integration-ci](#) 集成测试

同类系统对比：

系统	KV Cache 策略	论文
vLLM	PagedAttention (Block 分页)	SOSP 2023
SGLang	RadixAttention (Radix Tree 缓存)	NeurIPS 2024

参考资料

- 论文: [Efficient Memory Management for LLM Serving with PagedAttention \(SOSP 2023\)](#)
- 中文翻译: [references/vllm-project/pagedattention/paper_cn.pdf](#)
- 代码: [vllm-project/vllm](#)
- 项目摘要: [projects/vllm-project/vllm/summary.md](#)
- 跨资源查询: `by-tag['vllm']` → 6 项目 + 1 论文